

AgriSight-AI

Complete Project Guide

How it works · The codebase, file by file · Every technology and its job · All features · What makes it different · Real results · 50 viva questions answered (+15 bonus)

B.Tech (CSE) Micro Project · Bharati Vidyapeeth's College of Engineering, Delhi
Team: Sparsh Bhardwaj, Pranshu Garg, Aditya Tyagi · Guide: Ms. Bhawna Shukla

How this guide was made

This guide was written by reading the actual code in the **MicroProject** folder (the Python ML service, the Node backend, the React frontend, the docs and the saved model reports). Every number in it comes from the project's own report files, such as `models/evaluation_report.json` and `models/seg_evaluation_report.json`.

Where the code and your earlier explainer PDF disagree, this guide follows the code and flags the difference in **Section 7**. Read that section before your viva.

Contents

#	Section	What you get
1	How the project works	The big picture, the two ways to use the app, and the 12-step journey of one photo
2	The codebase	Folder map, the 3 services, key files and what each one does, one request followed through the code
3	Technologies and their jobs	Every library and tool, grouped by layer, with a plain-words explanation
4	Features	All the features, including Farmer Mode, voice, Hindi, offline mode, plots, weather risk and expert review
5	How it differs from other projects	A side-by-side comparison with typical plant-disease projects
6	Real results	Measured accuracy, Dice and IoU, weakest classes, and what has not been measured yet
7	Corrections to the old explainer	The places where the code differs from the earlier PDF (important for the viva)
8	The 50 viva questions, answered	Improved answers based on the code, with the real numbers
9	15 bonus questions	Questions about the parts the old PDF did not cover

Key facts to memorise

Fact	Value (from the project's own files)
Classifier	MobileNetV2 (transfer learning, ImageNet weights), 224×224 input, 38 classes, 14 crops
Dataset	PlantVillage: 54,305 images, split 70 / 15 / 15 (train / validation / test), seed 42
Test accuracy	98.49% (8,022 of 8,145 test images correct). Weighted precision 98.51%, recall 98.49%, F1 98.49%
Segmentation model	U-Net with a MobileNetV2 encoder, 256×256, trained on a Kaggle leaf-lesion dataset (588 image+mask pairs, split 412/88/88)
Segmentation score	Mean Dice 0.782 , micro Dice 0.806, mean IoU 0.663, pixel precision 0.844, recall 0.772
Confidence tiers	HIGH ≥ 0.80 · LOW 0.60–0.80 · UNRELIABLE < 0.60 (set in .env)
RAG	38 extension-guide PDFs → 800-character chunks (120 overlap) → all-MiniLM-L6-v2 embeddings → Chroma. Hybrid search (embeddings + BM25 keywords), top 5, minimum relevance 0.35
LLM	llama3.2 running locally through Ollama, temperature 0.2, strict 7-rule system prompt, 5 fixed headings
Faithfulness check	NLI model cross-encoder/nli-deberta-v3-small. A sentence is kept only if some evidence chunk supports it with probability ≥ 0.5
Tests	Over 250 automated tests across the 3 services (ML pytest, backend node:test, frontend vitest)

1. How the Project Works

1.1 The idea in plain words

A farmer photographs a sick leaf. AgriSight-AI tells them four things: **which disease** it most likely is, **how sure** it is, **where on the leaf** the damage is and how much of the leaf is affected, and **what reliable documents say** about the disease, with the source named. When it is not sure, or cannot find a document that supports an answer, it says so instead of guessing.

Think of it as a small team of specialists, each doing one job and checking the others:

- **The eye (CNN):** a neural network trained on tens of thousands of leaf photos names the disease.
- **The honesty meter (confidence tiers):** decides whether the eye is sure enough to be trusted.
- **The pointer (Grad-CAM):** shows which part of the photo the eye looked at.
- **The painter (U-Net segmentation):** colours every diseased pixel red and measures how much of the leaf is affected.
- **The librarian (RAG):** finds the most relevant paragraphs in a library of agricultural guides.
- **The writer (local LLM):** turns those paragraphs into simple advice, and is only allowed to use them.
- **The fact-checker (NLI model):** reads each sentence the writer produced and deletes any sentence the paragraphs do not support.

1.2 Two ways to use the app

	Farmer Mode (no login)	Account Mode (login)
Who it is for	A farmer with a basic phone who may not read well	A farmer, student or agronomist who wants history and detail
How you start	Big camera button. The phone checks each frame and takes the photo automatically when it is sharp, bright and centred on a leaf	Upload or capture a photo on the Diagnose page
What you get	A result read out loud in Hindi or English from pre-written, reviewed scripts, a shopkeeper card, and a list of nearby agri shops and offices	Full result: confidence gauge, alternatives, Grad-CAM, lesion mask with severity, a cited explanation, and the live pipeline progress
Extras	Works offline as a “quick check” using an on-phone copy of the model	Plots (fields), a timeline of scans, actions and reminders, re-scan comparison, weather-based risk, expert review queue
API route	/api/farmer/predict	/api/predict (needs a JWT login token)

1.3 The journey of one photo (as implemented in pipeline.py)

The ML service runs the photo through named stages: validate → classify → explain → retrieve → generate → verify → translate. Each stage sends a live progress event (Server-Sent Events), so the progress bar on screen is real rather than a timer.

1	Capture and upload	On the phone, frameQuality.js checks brightness, sharpness, how leaf-like the frame is, and motion at about 3 frames per second. After 4 good frames in a row it takes one photo. The photo goes to the Node backend, which checks the file type and size and forwards it to the Python service with a secret internal token.
---	---------------------------	---

2	Image validation	image_validation.py: file 8 MB or smaller, JPEG/PNG/WebP only, at least 64×64 pixels, and a sharpness check (variance of the Laplacian must be ≥ 25). A bad photo is rejected with a clear message (HTTP 422).
3	Classification	The image is resized to 224×224 and normalised with ImageNet mean and std, then MobileNetV2 gives 38 scores (logits). Softmax (optionally divided by a temperature T) turns them into probabilities.
4	Confidence tiering	confidence.py: top probability ≥ 0.80 is HIGH; 0.60–0.80 is LOW (it says it cannot separate disease A from disease B); below 0.60 is UNRELIABLE (“please upload a clearer photo”).
5	Differential diagnosis	Up to 2 other diseases whose probability is at least 10% are shown as alternatives.
6	Out-of-distribution gate	ood.py: a leaf-colour check (not_a_leaf) and an energy score check (unsupported_crop) can downgrade any result to UNRELIABLE. These switch on only after the thresholds are fitted (see Section 7).
7	Grad-CAM	gradcam.py hooks the last convolutional block, back-propagates the top class score, weights each feature map by its average gradient, applies ReLU, normalises, and overlays a heatmap on the photo.
8	Lesion segmentation	segmentation/inference.py: the U-Net paints diseased pixels red. The leaf area is estimated with an Otsu threshold on colour saturation. Severity = lesion pixels ÷ leaf pixels × 100, labelled mild (under 10%), moderate (10–25%) or severe (over 25%). Skipped for healthy or unreliable results.
9	Safety stop	If the result is UNRELIABLE, the pipeline stops here. No retrieval and no LLM run (retrieval_status = skipped_low_confidence).
10	Retrieval (RAG)	retrieval.py builds the query “<crop> <disease> symptoms causes management treatment + possible differential”. hybrid.py ranks chunks by $0.6 \times \text{embedding similarity} + 0.4 \times \text{BM25 keyword score}$. Only chunks with embedding relevance ≥ 0.35 pass. If none pass, the status is insufficient_evidence.
11	Grounded generation	llm_service.py sends the evidence to llama3.2 on Ollama (temperature 0.2) with 7 strict rules and 5 fixed headings. With no evidence, the LLM is told to say reliable information was not found.
12	Faithfulness check	faithfulness/checker.py tests every sentence in the three fact sections against every evidence chunk with an NLI model. Sentences with entailment below 0.5 are removed, and a visible note says how many were removed.
13	Translation, logging, response	Hindi translation (IndicTrans2 or Bhashini; English stays the source of truth). A privacy-safe line goes to predictions_log.jsonl for Power BI. The backend stores the scan and the frontend shows everything.

Say this in one breath

"A photo is checked for quality, classified by a specialised CNN, sorted into an honest confidence tier, explained with Grad-CAM, and measured with a U-Net lesion mask. Then only trusted document text is retrieved, a local LLM explains it, and an NLI fact-checker deletes any sentence the documents don't support. If the system is unsure at any stage, it says so."

2. The Codebase and How It Works

2.1 Architecture: three services, one rule

The project is split into three programs that talk over HTTP. The rule, written in docs/architecture.md, is: **React only displays, Node handles web concerns (login, limits, database), and Python makes every ML, RAG and LLM decision.** The Python service rejects any request without the shared X-Internal-Token header, so nobody can skip the backend's login and rate limits by calling Python directly.

```
Phone / Browser (React 19 PWA, Vite)
  | HTTPS: photo upload, JSON, Server-Sent Events
  v
Node.js + Express gateway (port 5000-ish)
  auth (JWT + refresh cookie) · roles · zod validation · rate limits
  SQLite/Postgres via Knex · EXIF-stripping image store
  help finder (OpenStreetMap) · weather risk rules · audio clips
  | HTTP + X-Internal-Token
  v
Python FastAPI ML service (port 8000)
  validation · MobileNetV2 · confidence/OOD · Grad-CAM · U-Net
  Chroma + MiniLM + BM25 · Ollama llama3.2 · NLI check · translation
  |
  +--> Ollama server (localhost:11434)   +--> Chroma DB on disk
```

2.2 Folder map

Folder	What lives there
ml-service/app/	All the AI. Sub-folders below.
training/	dataset.py (loads PlantVillage, 70/15/15 split, augmentation), model_factory.py (MobileNetV2 / EfficientNet-B0 with a new 38-class head), train.py (2-phase training), evaluate.py
inference/	service.py (loads the model once, predicts), confidence.py (tiers + alternatives), gradcam.py, image_validation.py, ood.py
calibration/	temperature.py (temperature scaling, ECE, reliability diagram), fit.py (fits T and the OOD thresholds), synthetic_junk.py (fake non-leaf images for testing)
segmentation/	model.py (U-Net), losses.py (BCE + Dice), metrics.py (Dice, IoU, boundary F1), train.py, evaluate.py, prepare.py, inference.py (mask + severity)
rag/	pdf_ingestion.py (pypdf text per page), chunking.py, vector_store.py (Chroma), ingest.py, retrieval.py, hybrid.py (BM25 + embeddings), coverage.py (which diseases have documents)
services/	llm_service.py (Ollama prompt + call), prediction_log.py (JSONL analytics log)
faithfulness/	checker.py (NLI sentence check), evaluate.py (aggregate unsupported-claim rate)
symptoms/	questions.py: yes/no symptom questions with a Bayes update for close cases
translation/	IndicTrans2 / Bhashini / none backends, and a drift check
export/, evaluation/	onnx_export.py (model for the phone), robustness.py (blur, low light, JPEG, rotation, occlusion tests)
api/, pipeline.py, main.py	FastAPI routes (predict, stream, refine, status, metrics), the stage pipeline, and app start-up
backend/src/	app.js and server.js, routes/ (auth, farmer, predict, account, expert, admin, status), middleware/ (auth, upload, rate limits, validation), services/ (mlClient, sseRelay, riskEngine, weather, shops, helpCenters, exif, followup, metrics), db/ (Knex migrations)

Folder	What lives there
frontend/src/	pages/ (Landing, Login, Register, account/*), farmer/ (camera, capture state machine, safety gating, audio, help finder), components/ (DiagnosisCard, GradCamView, ConfidenceGauge, PipelineTrail, ...), offline/ (ONNX classifier, IndexedDB queue, PWA), i18n/ (en, hi)
knowledge_base/	documents/ (38 PDFs + .meta.json sidecars with title/organisation/URL), questions/ (symptom question sets), risk_rules/ (weather rules with citations)
models/	plant_disease_model.pt (9.3 MB), lesion_seg_model.pt (10.5 MB), configs, evaluation and training reports
audio_scripts/, audio_clips/	Pre-written spoken scripts and generated audio for every class, in English and Hindi
tests/, backend/tests, frontend/src/__tests__	Automated tests for all three services
docs/	architecture, api, evaluation, segmentation, rag, model card, privacy, assumptions, HUMAN_TODO

2.3 The most important code, explained

(a) Confidence tiers: inference/confidence.py

```

if top.probability < LOW_CONFIDENCE_THRESHOLD:      # 0.60
    level = "unreliable" -> "Please upload a clearer, well-lit photo"
elif top.probability < HIGH_CONFIDENCE_THRESHOLD:  # 0.80
    level = "low" -> "cannot reliably distinguish A and B"
else:
    level = "high"
alternatives = next classes with probability >= 0.10 (max 2)

```

The file deliberately contains **no disease-to-treatment table**. It only decides how confident the system is. All advice must come from the documents.

(b) Two-phase transfer learning: training/train.py

- Phase 1: the MobileNetV2 backbone is frozen and only the new 38-class Linear head is trained (Adam, learning rate 1e-3).
- Phase 2: the last 20 layers are unfrozen and fine-tuned at one tenth of the learning rate (1e-4). Early stopping has patience 4.
- Training-only augmentation: horizontal flip, rotation $\pm 15^\circ$, colour jitter (brightness, contrast and saturation ± 0.2).
- The test split is used only once, at the very end, to write evaluation_report.json.

(c) Grad-CAM: inference/gradcam.py. Forward and backward hooks on the last feature block capture activations A_k and gradients. Weight w_k = average of the gradient over the map. CAM = $\text{ReLU}(\sum w_k \cdot A_k)$, normalised to 0-1, resized to 224×224, and blended at 45% with a jet-style colour map. No OpenCV or matplotlib is needed.

(d) U-Net: segmentation/model.py. The encoder is the MobileNetV2 feature extractor, tapped at strides /2, /4, /8, /16 and /32 (16, 24, 32, 96 and 320 channels). The decoder has 4 up-blocks. Each one upsamples, **concatenates the matching encoder feature map (the skip connection)**, and applies two conv-BN-ReLU layers. A final 1×1 conv outputs one channel, a per-pixel “lesion” logit. The loss is 0.5·BCE + 0.5·soft-Dice (losses.py), because lesions are small and BCE alone lets the model ignore them.

(e) Hybrid retrieval with a safety gate: rag/hybrid.py + retrieval.py

```

relevance = 1 / (1 + squared_L2(query_embedding, chunk_embedding))
hybrid     = 0.6 * minmax(relevance) + 0.4 * minmax(BM25 keyword score)
take top-5 by hybrid score
keep only chunks with relevance >= 0.35          # the safety gate
none left -> status "insufficient_evidence"      # LLM told to say "not found"

```

BM25 can change the order of chunks, but it can never let an irrelevant chunk through. The gate uses the meaning-based score only.

(f) The LLM prompt: `services/llm_service.py`. The 7 rules are: use only the retrieved evidence; never invent citations; do not claim certainty when confidence is low; no pesticide products or doses unless they are in the evidence; if the evidence is missing, say so and refer to an expert; you do not diagnose the image; use simple language. The output must use the headings: *What is happening?* · *Why does the system think this?* · *What can cause or spread it?* · *What should you consider doing?* · *Important caution*.

(g) Faithfulness: `faithfulness/checker.py`. It splits the three factual sections into sentences. For each sentence it computes P(entailment) against every evidence chunk and keeps the highest value. If that value is below 0.5, the sentence is removed (FAITHFULNESS_ACTION=remove) or flagged (=flag). The response reports unsupported_rate.

2.4 Following one request through the code

#	File	What happens
1	frontend/src/pages/account/DiagnosePage.jsx → api/sse.js	Sends the photo as multipart form data and opens a Server-Sent Events stream
2	backend/src/routes/predict.js	requireAuth (JWT), multer upload limits, rate limit, then forwards the request to the ML service
3	backend/src/services/sseRelay.js	Relays stage events to the browser and saves the scan to the database when the final result event arrives
4	ml-service/app/security.py	Checks X-Internal-Token
5	ml-service/app/api/predict.py → pipeline.run_prediction()	Runs validate → classify → explain → retrieve → generate → verify → translate
6	inference/service.py	Singleton model, preprocessing, softmax/T, build_diagnosis, OOD gate, Grad-CAM
7	segmentation/inference.py	Mask, severity %, severity band
8	rag/retrieval.py, services/llm_service.py, faithfulness/checker.py	Evidence, explanation, sentence check
9	services/prediction_log.py	Writes one privacy-safe JSON line for analytics
10	components/DiagnosisCard, GradCamView, SegmentationMetricsCard, SourcesList, PipelineTrail	Displays the result

3. Technologies Used and What Each One Does

Layer	Technology	Its job in this project
Frontend	React 19	Builds the user interface from reusable components (DiagnosisCard, GradCamView and so on)
	Vite 8	Development server and production bundler (fast hot-reload, builds /assets)
	react-router-dom 7	Pages: landing, login, register, farmer, account pages
	i18next + react-i18next	English and Hindi interface text (locales/en.json, hi.json)
	onnxruntime-web (WASM)	Runs the exported model inside the phone's browser for the offline quick check
	Service Worker + Web App Manifest (PWA)	Makes the site installable and caches the app, model and audio for offline use
	IndexedDB	Queues photos taken offline, then sends them when the phone is back online
	getUserMedia / Web Speech API	Rear camera access; speech synthesis as a fallback when an audio clip is missing
Backend	Node.js + Express 4	API gateway: routes, error handling, forwarding to Python
	jsonwebtoken + bcryptjs	Login: 15-minute access tokens, rotating refresh cookie, passwords hashed with bcrypt (cost 12)
	Knex + better-sqlite3 / pg	Database layer with migrations. SQLite for development, PostgreSQL for Docker
	multer, zod, helmet, express-rate-limit, cors, morgan	Upload handling, input validation, security headers, rate limiting, request logs
	OpenStreetMap Overpass / Nominatim, Open-Meteo	Nearby agri shops and village search; hourly weather forecast for the risk rules
ML service	Python + FastAPI + Uvicorn + Pydantic	Fast async API with automatic Swagger docs at /docs and typed response schemas
	PyTorch + torchvision	Training and running MobileNetV2 and the U-Net
	MobileNetV2 (EfficientNet-B0 alternative)	Light CNN backbone (about 3.5 M parameters) pretrained on ImageNet
	Grad-CAM (hand-written)	Visual explanation heatmap
	U-Net (hand-written, MobileNetV2 encoder)	Pixel-level lesion mask. Scored with Dice, IoU and boundary F1
	NumPy, Pillow, scikit-learn	Array maths, image handling, metrics (precision, recall, F1, confusion matrix)
RAG	pypdf	Extracts text page by page from the PDFs, so citations can give page numbers
	Sentence Transformers: all-MiniLM-L6-v2	Turns text into 384-number vectors that capture meaning
	Chroma	Vector database stored on disk (data/vector_db/chroma.sqlite3)
	rank_bm25	Keyword relevance (BM25) for hybrid search
LLM	Ollama + llama3.2	Local open-weight LLM server. Writes the explanation. Free, offline, private
Verification	cross-encoder/nli-deberta-v3-small	Natural Language Inference: does the evidence entail this sentence?

Layer	Technology	Its job in this project
Translation	IndicTrans2 (AI4Bharat) / Bhashini	English → Hindi for explanations. The LLM is never asked to write Hindi
Audio	espeak-ng (demo) / Coqui / recorded clips	Spoken results for Farmer Mode
Deployment and ops	Docker Compose (4 services), .env	frontend (nginx), backend, ml-service, postgres. Every threshold is an environment variable
Analytics	JSONL → CSV → Power BI	Dashboard of predictions, confidence, retrieval status
Testing	pytest, node:test + supertest, vitest + Testing Library, oxlint	Automated tests and linting

4. Features of the Project

4.1 Core diagnosis features

- **38-class disease classification** across 14 crops (apple, blueberry, cherry, corn, grape, orange, peach, bell pepper, potato, raspberry, soybean, squash, strawberry, tomato), including “healthy” classes.
- **Image quality gate**: size, type, resolution and blur checks before any AI runs.
- **Three-tier confidence** (high / low / unreliable) with an honest message for each tier.
- **Differential diagnosis**: up to 2 alternatives with probability $\geq 10\%$. In the LOW tier it names the two diseases it cannot separate.
- **“I don't know” gate (out-of-distribution)**: rejects non-leaf photos and unfamiliar leaves with reason codes. The mechanism is built and switches on once it is fitted.
- **Grad-CAM heatmap** with an on-screen note that it shows correlation, not proof.
- **Lesion segmentation**: a red mask of the diseased area plus a **severity %** and a band (mild / moderate / severe).
- **Adaptive symptom questions**: when the top two diseases are close (gap under 0.30), the farmer answers 2–3 cited yes/no questions (for example early vs late blight). A **Bayes-rule update** re-weights the two probabilities.
- **Evidence-grounded advice (RAG)** with the title, organisation and page number of every source.
- **Sentence-level fact-check** that removes unsupported claims.
- **Live pipeline progress** through real Server-Sent Events, stage by stage.

4.2 Farmer Mode (made for low literacy and low connectivity)

- **Guided auto-capture camera**: checks on the phone for brightness, Laplacian sharpness, leaf-likeness and motion, then takes the photo automatically after 4 good frames. There is also a manual shutter and a gallery fallback. Video is never streamed.
- **Voice-first results** in Hindi or English, from **pre-written scripts only**. Live LLM text is never spoken.
- **Safety gating** (safetyGating.js): HIGH → name + only document-cited safe steps; LOW → “not sure”, both candidates and a caution; UNRELIABLE → never says a disease name; no evidence → “reliable advice was not found, visit the agriculture office”.
- **Shopkeeper card**: crop and disease in English and the farmer's language, the confidence band and the photo, so the farmer can show it at a shop. It deliberately recommends no product.
- **Nearby help finder**: agri-input shops from OpenStreetMap (15 km radius), plus verified agriculture offices and KVKs. Location is rounded to about 110 m for privacy, and the farmer must give consent.
- **Offline quick check**: the model runs in the browser (ONNX). The result is always labelled “offline quick check”, gives no advice, and the photo is queued for full analysis later.

4.3 Account Mode

- **Secure accounts**: register, login, logout after 5 failed attempts, rotating refresh tokens with reuse detection, and user / expert / admin roles.
- **Plots and timeline**: every scan belongs to a field. Photos are stored with EXIF (GPS) metadata removed and deleted after 180 days.
- **Actions → reminders → follow-up**: log what you did, get a re-scan date, and compare the new scan with the old one (labelled “indicative”).

-
- **Weather risk indicator:** a rules engine (not ML) checks the Open-Meteo forecast against disease-favourable conditions quoted from a cited document. It uses only numbers that appear in the quote.
 - **Expert review queue:** experts confirm or correct labels. The admin monitoring page then shows accuracy against the expert labels.
 - **Dispute a scan, delete a scan, delete an account** (privacy by design).
 - **Admin monitoring:** scan counts, confidence mix, retrieval outcomes, segmentation report.

4.4 Engineering and quality features

- Every threshold is configurable in .env, so nothing is hard-coded.
- Robustness test suite: blur, low light, JPEG compression, rotation, occlusion and synthetic junk images.
- ONNX export with a parity check (the largest logit difference from PyTorch was 7.6×10^{-6}) and an int8 version (2.5 MB instead of 9.1 MB).
- A knowledge-base coverage report showing which diseases have supporting documents.
- Privacy-safe analytics log: no image, no IP address, no location.
- Graceful degradation: if Ollama is down, you still get the diagnosis, Grad-CAM, mask and sources; only the written explanation is missing.

5. How AgriSight-AI Differs from Other Plant Disease Projects

Most student and many published projects follow the pattern “train a CNN on PlantVillage → show a label and a percentage”. Some newer ones add “ask ChatGPT for a cure”. AgriSight-AI goes further on trust, completeness and real-world use.

Aspect	Typical project	Chatbot-style project	AgriSight-AI
Output	Label + %	Free-text guess	Label, tier, alternatives, heatmap, lesion mask, severity %, cited advice
Uncertainty	Always gives an answer	Sounds confident	HIGH / LOW / UNRELIABLE tiers, OOD “not a leaf” gate, stops before advice when unreliable
Explainability	None	None	Grad-CAM (where it looked) and U-Net mask (where the disease is)
Severity	No	No	Severity % and band from pixel segmentation, evaluated with Dice/IoU
Treatment advice	Hard-coded table or none	From LLM memory (can hallucinate)	RAG from 38 extension PDFs with page citations, and an explicit “insufficient evidence” outcome
Hallucination control	n/a	None	Strict prompt + relevance gate + NLI sentence-level fact-check
Cost and privacy	n/a	Paid cloud API; data leaves the device	Local Ollama LLM: free, offline-capable, private
Ambiguous cases	Picks one	Picks one	Asks cited symptom questions and updates with Bayes' rule
Users	Tech-literate	Tech-literate	Voice-first Hindi Farmer Mode, auto-capture camera, shopkeeper card, nearby help
Connectivity	Needs internet	Needs internet	PWA + on-device ONNX quick check + offline queue
Follow-up	One-off	One-off	Plots, timeline, reminders, re-scan comparison, weather risk rules
Human oversight	None	None	Expert review queue and accuracy against expert labels
Engineering	One notebook	One script	3 services, auth, rate limits, Docker, 250+ tests, docs, model card

The three-line answer to “Why is yours different?”

- 1. Safety first:** it prefers refusing to answer over being confidently wrong, at every layer.
- 2. Measured, not claimed:** every model has a written evaluation report (accuracy/F1 for the CNN; Dice/IoU/boundary F1 for the U-Net).
- 3. Built for the real user:** an Indian smallholder farmer: voice, Hindi, low literacy, weak network, needs a shop or an office.

6. Real Results from the Project Files

6.1 Disease classifier (models/evaluation_report.json)

Metric	Value
Test images	8,145 (15% held-out split, never used in training)
Accuracy	98.49% (8,022 correct)
Precision / Recall / F1 (weighted)	98.51% / 98.49% / 98.49%
Validation accuracy after phase 1 (head only)	94.59%
Validation accuracy after phase 2 (fine-tuning)	98.49%

Weakest classes (lowest F1): Corn Cercospora / Gray leaf spot 0.895, Potato healthy 0.927 (recall 0.86, and only 152 training-set images, so class imbalance), Tomato early blight 0.929, Peach healthy 0.939, Corn Northern leaf blight 0.945.

Most common confusions: Tomato Target spot → Spider mites (10 images), Corn Northern leaf blight ↔ Gray leaf spot (8 + 7), Peach bacterial spot → healthy (7), Tomato late blight → early blight (6). These are visually similar leaf-spot diseases, which is exactly why the project has the differential list and the symptom questions.

6.2 Lesion segmentation (models/seg_evaluation_report.json)

Metric (88 test images, 256×256, threshold 0.5)	Value
Mean Dice (each image counts equally)	0.782
Micro Dice (all pixels pooled)	0.806
Mean IoU / Micro IoU	0.663 / 0.675
Pixel precision / recall	0.844 / 0.772 (it misses more lesion pixels than it wrongly adds)
Boundary F1 (2-pixel tolerance)	0.633
Dice by lesion size	small (under 5%) 0.724 · medium 0.797 · large (over 20%) 0.796
Median Dice / worst image	0.833 / 0.218
Training	15 epochs with the encoder frozen, then 7 fine-tuning epochs. 588 pairs split 412 / 88 / 88

A mean Dice of 0.78 sits inside the 0.72–0.86 range that published studies report for realistic or lightweight leaf-lesion models. Small lesions are the hardest, as expected.

6.3 Built but not yet measured (be honest about these)

Item	Status in the code
Temperature scaling (ECE) and the OOD gate	Code and tests are complete, but not fitted : models/model_config.json has no “temperature” or “ood” block, so $T = 1$ and the OOD gate is off. Run <code>python -m app.calibration.fit</code> .
RAG evaluation (% relevant, % correct citation)	Method is documented in docs/evaluation.md; manual run pending
Unsupported-claim rate (NLI)	Computed for each response; aggregate not yet measured
int8 ONNX accuracy	Not measured. The fp32 model is used offline for that reason
Robustness suite, symptom-question benefit	Scripts ready; numbers pending

Be careful in the viva

Your old PDF said that noise was classified with 86.1% confidence “before calibration was applied”. In the current saved model, calibration has **not** been applied yet. Say: “We observed this overconfidence, so we built temperature scaling and an energy-based OOD gate. The fitting script is ready and is the next step.” Better still, run the fit script before the viva and quote the ECE before and after.

7. Where the Code Differs from Your Old Explainer PDF

An examiner may open the repository. Make sure what you say matches what is there.

Old PDF said	What the code actually shows	What to say
Knowledge base = ICAR / FAO documents; only a couple of sample PDFs	38 PDFs , mostly US university extension guides (Purdue, Cornell, Kentucky, Wisconsin, Kansas State, Virginia Tech, Arkansas, Hawaii and others). The two tomato “guide” PDFs are placeholders.	“38 curated university extension guides with metadata. ICAR/state-agriculture documents are the planned addition for Indian conditions.”
Calibration was applied	Mechanism exists; not fitted yet (no temperature in model_config.json)	“Built and unit-tested, fitting is pending.”
Retrieval = embedding search only	Hybrid : 0.6 embedding + 0.4 BM25, a 0.35 relevance gate, and an optional cross-encoder re-ranker	Mention hybrid search: it catches exact disease names that embeddings can blur.
U-Net (generic)	U-Net with a pretrained MobileNetV2 encoder , BCE + Dice loss, trained on the Kaggle leaf-disease segmentation set	Quote Dice 0.78 / IoU 0.66.
Dice from 0.72 to above 0.95	Your own measured value is 0.782 (mean) / 0.806 (micro)	Always give your own number first, then the literature range.
Segmentation runs on every image	Skipped for healthy or unreliable results, or if the model file is missing	“It only runs when there is a disease to measure.”
Faithfulness checks every sentence	Checks sentences in the 3 factual sections; the “why the system thinks this” and “caution” sections are about the system, so they are not checked	Explain the reason.
Features: web upload only	Also Farmer Mode, voice, Hindi, offline ONNX, shops and offices, plots, reminders, weather risk, expert review, SSE, auth	Use Section 4; these are strong novelty points.
Training details	The saved run used 1 epoch per phase (the default is up to 15 with early stopping) and still reached 98.49% test accuracy	If asked about epochs: “1+1 in the saved run; transfer learning converges fast on PlantVillage.”

8. The 50 Viva Questions, Answered

These are improved answers based on the code. Short enough to say aloud, with the real numbers where they help.

A. Project basics and motivation

Q1. What problem does your project solve?

Farmers often cannot get an expert to identify a leaf disease quickly. Our system takes a leaf photo and returns the likely disease, an honest confidence level, a heatmap of where the model looked, a mask and percentage of the affected area, and advice taken only from cited agricultural documents. When it is unsure, it says so.

Q2. Why is this problem important?

Plant diseases cause large yield losses every year. A wrong or late diagnosis leads to the wrong pesticide, wasted money and chemical overuse. This hurts smallholder farmers most, and they are the people with the least access to agronomists.

Q3. What makes your project different from just asking ChatGPT?

ChatGPT gives a fluent guess that cannot be measured, explained at the pixel level, or traced to a source. We use a specialised CNN with a measured 98.49% test accuracy, a confidence tier with a refusal option, Grad-CAM and segmentation for visual proof, and a local LLM that may only use retrieved document text. An NLI model then deletes any sentence the documents do not support.

Q4. Who are the intended users?

Mainly smallholder farmers, through the voice-first Hindi/English Farmer Mode that needs no login. Also home gardeners, extension workers and agronomists (Account Mode and the expert review role), and agriculture students.

Q5. Is it meant to replace expert agronomists?

No. It is a decision-support tool. Every response carries a caution. Unreliable or low-confidence results send the farmer to the agriculture office, and experts can review and correct scans in the expert queue.

Q6. Which SDGs does it align with?

SDG 2 Zero Hunger (less crop loss), SDG 9 Industry, Innovation and Infrastructure (applied AI), SDG 12 Responsible Consumption (no product pushing, less unnecessary pesticide), SDG 15 Life on Land (healthier ecosystems). The Hindi, voice and offline features also support SDG 10 Reduced Inequalities.

Q7. What dataset did you use?

For classification: **PlantVillage**, 54,305 labelled leaf images in 38 classes across 14 crops, split 70/15/15 with a fixed seed of 42.

For segmentation: a **Kaggle leaf-disease segmentation dataset** with hand-drawn lesion masks (588 pairs). PlantVillage has no masks, so a second dataset was needed.

B. CNN and disease classification

Q8. Which model do you use for classification?

MobileNetV2 pretrained on ImageNet, with its last layer replaced by a 38-class Linear layer. EfficientNet-B0 can be switched in with one setting (BACKBONE). MobileNetV2 is light (about 3.5 M parameters, a 9.3 MB file), fast on CPU, and small enough to run in a phone browser.

Q9. What is transfer learning and why did you use it?

We reuse a network that already learned general visual features (edges, textures, shapes) from millions of ImageNet images. We train it in two phases: first only the new head with the backbone frozen (learning rate 1e-3), then the last 20 layers at learning rate 1e-4. It needs far less data and time, and reached 98.49% test accuracy.

Q10. What is a CNN and how does it “see” an image?

A Convolutional Neural Network slides small learnable filters over the image. Early layers detect edges and colours, and deeper layers combine them into textures, spots and lesion shapes. Pooling and striding shrink the map, and a final classifier turns these features into one score per class. Softmax turns the scores into probabilities.

Q11. What preprocessing do you apply?

Resize to 224×224, convert to a tensor, and normalise with the ImageNet mean and standard deviation (because the backbone was pretrained on ImageNet). During training only, we also apply a random horizontal flip, rotation $\pm 15^\circ$ and colour jitter $\pm 20\%$. Before any of this, the validation step rejects files that are too big, the wrong type, smaller than 64 px, or blurry.

Q12. What is confidence tiering and what are the thresholds?

The top probability is put into a tier: HIGH if ≥ 0.80 , LOW if between 0.60 and 0.80 (“cannot reliably distinguish A and B”), UNRELIABLE if below 0.60 (“upload a clearer photo”, and the pipeline stops before RAG and the LLM). The thresholds are environment variables, so a deployment can choose its own risk level.

Q13. What is differential diagnosis in your system?

Besides the top class we show up to 2 alternatives with probability of at least 10%. If the top two are within 0.30 of each other and a cited question set exists, the farmer is asked yes/no symptom questions, and Bayes' rule updates the two probabilities.

Q14. How do you evaluate the classifier?

On the 15% held-out test split (8,145 images) that was never used in training: accuracy 98.49%, weighted precision 98.51%, recall 98.49%, F1 98.49%, plus per-class metrics and a 38×38 confusion matrix saved in `evaluation_report.json`.

Q15. What is a confusion matrix and why is it useful here?

It is a table of true class against predicted class. The diagonal shows correct predictions and the off-diagonal cells show mistakes. It shows which diseases get mixed up. For us the biggest confusions were Tomato target spot → spider mites (10), Corn northern leaf blight ↔ gray leaf spot (15 in total), and Tomato late → early blight (6), which is why we built the differential list and the symptom questions.

C. Grad-CAM and explainability

Q16. What is Grad-CAM?

Gradient-weighted Class Activation Mapping. We take the gradient of the predicted class score with respect to the last convolutional feature maps, average it to get one weight per map, compute a weighted sum of the maps, apply ReLU, normalise, and upscale it as a heatmap over the photo. Warm colours mark the regions that pushed the model towards its answer.

Q17. Why do you need Grad-CAM if the CNN gives a confidence score?

Confidence says how sure the model is, not why. A model can be 99% confident because it looked at the background or a watermark. Grad-CAM lets a user or expert check that the model focused on the lesions.

Q18. Does Grad-CAM prove the model's reasoning is correct?

No. It shows correlation (which pixels influenced the output), not agricultural correctness. Our UI states this in the response (GRADCAM_NOTE), and we added a separate segmentation model for an actual, evaluated map of where the disease is.

Q19. What problem does Grad-CAM address in AI generally?

The “black box” problem: deep networks give decisions without reasons, and that blocks trust in high-stakes fields like agriculture and medicine.

Q20. Could you use a different explainability method?

Yes: LIME, SHAP, Integrated Gradients, Grad-CAM++ or Score-CAM. Grad-CAM needs only one forward and one backward pass, needs no change to the architecture, and works on any CNN's last conv layer, so it is fast enough to run on every request. We implemented it by hand with PyTorch hooks.

D. Segmentation and the Dice coefficient**Q21. What is image segmentation, and how is it different from classification?**

Classification gives one label for the whole image. Segmentation gives a label for every pixel, here lesion or not lesion, so it answers “where” and “how much”, not just “what”.

Q22. What model do you use for segmentation?

A U-Net whose encoder is ImageNet-pretrained MobileNetV2 and whose decoder has 4 up-sampling blocks with skip connections. Input is 256×256 and the output is one lesion-probability channel, thresholded at 0.5.

Q23. How does U-Net work, in simple terms?

The encoder shrinks the image step by step (down to 1/32 of its size) to understand what is in it. The decoder enlarges it back to full size to draw the mask. Skip connections copy detailed features from each encoder level straight to the matching decoder level, so thin lesion edges are not lost. We tap MobileNetV2 at strides 2, 4, 8, 16 and 32.

Q24. What is the Dice Similarity Coefficient?

It measures the overlap between the predicted mask P and the expert mask G: $DSC = \frac{2|P \cap G|}{(|P| + |G|)} = \frac{2TP}{(2TP + FP + FN)}$. Our model scores **0.782 mean Dice** and **0.806 micro Dice** on 88 test images.

Q25. What does a Dice of 1 versus 0 mean?

1 means a perfect pixel-for-pixel match. 0 means no overlap at all. In our code, if both masks are empty the score is 1 (it correctly found no lesion), and if only one is empty the score is 0.

Q26. Why Dice instead of pixel accuracy?

Lesions are often a few percent of the image. A model that predicts “healthy everywhere” would still get about 97% pixel accuracy. Dice ignores the easy background and scores only the lesion region. That is also why our training loss includes a soft-Dice term, alongside BCE.

Q27. What real-world benefit does segmentation add?

Severity. We compute lesion pixels ÷ estimated leaf pixels × 100 and label it mild (under 10%), moderate (10–25%) or severe (over 25%). A 5% leaf and a 70% leaf have the same disease name but very different urgency. The leaf area comes from an Otsu threshold on colour saturation, so we call the result an estimate.

Q28. What Dice scores are considered good?

Published leaf-lesion work reports about 0.72–0.86 for realistic or lightweight models and above 0.95 on very clean curated sets. Our 0.78 mean / 0.81 micro is in the realistic range. By lesion size, small lesions (under 5% of the image) score 0.72 and medium/large about 0.80. We also report IoU (0.66) and boundary F1 (0.63).

E. RAG, the vector database and retrieval**Q29. What is Retrieval-Augmented Generation (RAG)?**

Before the LLM writes, we retrieve the most relevant passages from a curated document library and give only those to the LLM, telling it to base every fact on them. The answer is grounded in checkable sources rather than the model's memory.

Q30. Why not let the LLM answer from its training data?

LLMs hallucinate. Their knowledge may be outdated, from the wrong region, and has no citation. For pesticide or treatment advice a confident wrong answer costs a farmer money or a crop, so every claim must trace to a document, organisation and page.

Q31. What is a vector database, and which one do you use?

It stores text as embedding vectors and finds the stored vectors closest to a query vector, which gives search by meaning rather than exact words. We use Chroma, persisted on disk. Our score is $\text{relevance} = 1 / (1 + \text{squared L2 distance})$.

Q32. What embedding model do you use?

Sentence Transformers all-MiniLM-L6-v2. It maps any text to a 384-dimensional vector. It is small and fast on CPU, and texts with similar meaning get nearby vectors.

Q33. Walk me through retrieval step by step.

- 1) Ingestion (once): pypdf extracts text page by page. Each page is cut into 800-character chunks with 120 overlap (a chunk never crosses a page, so the page citation stays exact). Each chunk gets metadata from its .meta.json (title, organisation, URL), is embedded, and is stored in Chroma.
- 2) Query: "<crop> <disease> symptoms causes management treatment", plus the differential names.
- 3) Hybrid ranking: $0.6 \times \text{embedding relevance} + 0.4 \times \text{BM25 keyword score}$ (both min-max normalised). Keep the top 5.
- 4) Safety gate: drop chunks with embedding relevance below 0.35. The survivors go to the LLM with their citation labels.

Q34. What if retrieval finds nothing relevant?

The status becomes `insufficient_evidence` (or `knowledge_base_empty`). The LLM is explicitly told to say reliable information was not found and to recommend an expert, with no advice from general knowledge. In Farmer Mode the audio says "reliable advice was not found, please visit the agriculture office".

Q35. Where do your knowledge-base documents come from?

38 PDFs, mainly university extension publications (Purdue, Cornell, University of Kentucky, Wisconsin, Kansas State, Virginia Tech, Arkansas, Hawaii, Nebraska and others), each with a metadata sidecar. Two tomato-blight PDFs are placeholders and are refused in production mode. Adding ICAR and state agriculture department documents for Indian conditions is the next step. `coverage.py` reports which diseases still lack documents.

Q36. How do you evaluate retrieval?

With a manually verified set of test diagnoses. For each one we check: is the retrieved chunk relevant, is the cited source correct, and is the explanation grounded only in that evidence (% relevant, % citation-correct, % grounded). The NLI unsupported-claim rate gives an automatic signal on every response. The method is documented; the full manual run is pending.

F. LLM and faithfulness**Q37. Which LLM do you use, and why locally?**

llama3.2 through Ollama, at temperature 0.2 for steady, low-creativity output. Running it locally means no per-request cost, no internet needed at inference time, and no farmer data sent to a third party. The LLM never sees the image; it only explains the CNN's result.

Q38. What is “faithfulness”?

Whether every factual sentence in the explanation is actually supported by the retrieved evidence, rather than invented or added from the model's own memory.

Q39. How do you check faithfulness technically?

An NLI cross-encoder (cross-encoder/nli-deberta-v3-small) scores each sentence of the three factual sections against every evidence chunk. It keeps the highest P(entailment). If that is below 0.5, the sentence is removed and a note says “N statement(s) were removed...”, or it is flagged if configured that way. The response reports the unsupported rate.

Q40. What is the unsupported-claim rate and the target?

Unsupported sentences ÷ checked sentences. The target for the final output is effectively zero, because anything unsupported is removed. The rate before removal tells us how often the LLM tries to go beyond the evidence.

Q41. Can the faithfulness checker make mistakes?

Yes. NLI models can reject correct paraphrases or accept near-misses. We treat the score as an indicator, keep the threshold configurable, and back it up with manual RAG evaluation.

Q42. What if Ollama isn't running?

The pipeline catches LLMServiceError. The response still has the diagnosis, confidence tier, alternatives, Grad-CAM, lesion mask, severity and the retrieved sources; only the written explanation is missing. The health endpoint reports whether Ollama is reachable.

G. Architecture and tech stack**Q43. Describe your overall architecture.**

React PWA → Node/Express gateway (auth, validation, rate limits, database, help finder, weather rules) → Python FastAPI ML service (all ML/RAG/LLM), secured with an internal token. Ollama and Chroma sit beside the ML service. Stage events stream back through the backend to the UI as Server-Sent Events. Docker Compose runs frontend, backend, ml-service and Postgres.

Q44. Why split Node.js and Python?

Each language does what it is best at. Python has PyTorch, Sentence Transformers and Chroma. Node is excellent for HTTP, auth and I/O. The split keeps dependencies small, lets team members work independently, keeps the ML service off the public internet, and is a common production pattern.

Q45. How is the Power BI dashboard populated?

Every prediction appends one JSON line (timestamp, crop, disease, confidence, tier, alternatives, model version, retrieval status, with no image, IP or location) to predictions_log.jsonl. /api/analytics/export.csv converts it to CSV, and Power BI imports that together with evaluation_report.json.

H. Evaluation and metrics

Q46. What metrics does your project report?

Classification: accuracy, precision, recall, F1 (per class and weighted), confusion matrix. Segmentation: mean and micro Dice and IoU, pixel precision/recall, boundary F1, Dice by lesion size. RAG: % relevant, % citation-correct, % grounded. LLM: unsupported-claim rate. Calibration: ECE and a reliability diagram. OOD: false-accept rate on synthetic junk. Offline: fp32 vs int8 size and accuracy.

Q47. What is ECE and why does it matter?

Expected Calibration Error. Group predictions into 15 confidence bins, and in each bin compare average confidence with actual accuracy. ECE is the weighted average gap. It matters because our tiers depend on the numbers: if the model says 90%, it should be right about 90% of the time. Temperature scaling divides logits by a fitted T to fix this without changing the predicted class.

Q48. A noise image was classified with 86% confidence. What does that show?

Softmax classifiers are overconfident on inputs unlike anything they were trained on, because softmax must spread 100% over known classes. That is why we built temperature scaling and a two-part OOD gate: a leaf-colour ratio for “not a leaf”, and an energy score $E = -T \cdot \log \sum \exp(\text{logits}/T)$ with a threshold chosen so 95% of real validation leaves pass. The fitting script is ready; the fitted values are the next step.

I. Limitations and future scope

Q49. What are the biggest limitations?

1) PlantVillage is lab-style (plain backgrounds), so field accuracy will be lower. 2) Calibration and OOD are built but not yet fitted. 3) The knowledge base is mostly US extension material and needs Indian (ICAR/state) sources. 4) Grad-CAM shows correlation, not proof. 5) The severity leaf-area estimate uses a colour rule. 6) Hindi text and audio are machine drafts awaiting native-speaker review.

Q50. What would you build next?

Fit calibration and OOD and publish ECE; fine-tune on field images (for example PlantDoc or PlantSeg); add ICAR and Indian state documents and more languages; measure int8 accuracy for mobile; run RAGAS-style RAG evaluation; use expert-reviewed labels for periodic retraining; add human-recorded audio.

9. Fifteen Bonus Questions (on parts the old PDF skipped)

Q51. What is Farmer Mode's auto-capture?

The phone analyses about 3 frames per second at 160×120: brightness, sharpness (Laplacian variance), share of plant-coloured pixels in the centre, and motion. After 4 good frames in a row it takes one still photo. This runs on the phone and only decides when to take the photo; it does not diagnose anything.

Q52. Why are spoken results pre-written instead of reading out the LLM text?

Spoken advice to a farmer who cannot read has to be safe and reviewed. Live LLM or machine-translated text could contain an error that the farmer cannot check. The scripts are written once from cited documents and marked for native-speaker review.

Q53. How does safety gating work?

safetyGating.js turns the result into audio clip keys. HIGH → disease name + cited safe steps. LOW → “not sure”, both candidates and a caution. UNRELIABLE → never names a disease. No evidence → “reliable advice not found, visit the office”. Offline → “quick check only”.

Q54. How do the symptom questions use Bayes' rule?

$\text{posterior}(c) \propto \text{prior}(c) \times \prod P(\text{answer} | c)$. The prior is the CNN probability. $P(\text{yes} | c)$ comes from the cited question file, and “unsure” changes nothing. Only the two candidate classes are re-weighted, and their total probability is kept. The likelihoods are documented modelling constants (0.8 or 0.3), not invented “literature” numbers.

Q55. How does offline mode work?

onnx_export.py converts the PyTorch model to ONNX. onnxruntime-web runs it in the browser with the same temperature, tiers and OOD thresholds. Offline results are marked offline_not_checked and never give advice, and the photo is queued in IndexedDB for full analysis when the phone reconnects.

Q56. What is Server-Sent Events and why use it?

A one-way HTTP stream from server to browser. The pipeline emits validate, classify, explain, retrieve, generate, verify and translate events. The backend relays them, so the progress trail shows real stages instead of a fake timer.

Q57. How is authentication secured?

bcrypt-hashed passwords; a 15-minute HS256 JWT access token; a 48-byte random refresh token stored only as a SHA-256 hash in an httpOnly, SameSite=Strict cookie, rotated on every use (reuse revokes the whole family); logout after 5 failed logins for 15 minutes; generic error messages so attackers cannot tell which emails exist.

Q58. What privacy measures exist?

EXIF and GPS metadata stripped from stored photos; images deleted after 180 days; anonymous farmer scans (no image, IP or location stored); shop search location rounded to about 110 m and only with consent; analytics log contains no personal data; account and scan deletion.

Q59. How does the weather risk indicator work, and is it ML?

It is a rules engine, not ML. Each rule quotes a cited document (for example, conditions that favour late blight) and uses only the numbers in that quote. It checks the Open-Meteo hourly forecast and reports “favourable conditions forecast” or “no matching conditions” with the matching hours. It never calls this a prediction.

Q60. Why hybrid BM25 + embedding retrieval?

Embeddings capture meaning but can blur exact names (for example “Septoria” vs “Cercospora”). BM25 rewards exact keyword matches. Combining them (0.6 / 0.4) improves the ranking, while the embedding-only relevance gate still blocks irrelevant chunks.

Q61. Why BCE + Dice loss for the U-Net?

BCE alone is dominated by the many easy background pixels, so the model can score well while missing small lesions. Soft-Dice looks only at lesion overlap and pushes the model to find them. BCE keeps early training stable while the Dice gradients are noisy.

Q62. Mean Dice vs micro Dice: what is the difference?

Mean Dice averages each image's score, so every photo counts equally and small-lesion photos pull it down (0.782). Micro Dice pools all pixels first, so large lesions weigh more (0.806). Reporting both is more honest.

Q63. What is the energy score in OOD detection?

$E(x) = -T \cdot \log \sum \exp(\text{logit}_i/T)$ (Liu et al., 2020). Familiar inputs give low energy (large logits). The threshold is set on validation data so that 95% of real leaves pass; anything above it is marked `unsupported_crop`.

Q64. What does the expert review add?

Experts see a queue of scans and record a confirmed or corrected label. The admin monitoring page then reports model accuracy against these expert labels, which is a real-world accuracy signal beyond the PlantVillage test set. Retraining is not automatic, by design.

Q65. How was the project tested?

106 ML tests (pytest: dataset, model factory, Grad-CAM, confidence, validation, OOD/calibration, retrieval, faithfulness, segmentation metrics, API), 69 backend tests (auth, rate limits, EXIF, farmer, SSE, admin) and 84 frontend tests (Vitest: farmer flow, safety gating, i18n, offline, segmentation card), plus oxlint and an end-to-end smoke run with the real model.

Final tip

Good luck with your viva. If a number is not in this document, say “the evaluation script for that is built; the run is pending”. That answer is honest and it holds up under follow-up questions.